jplag.github.io/JPlag/comparison/22L-7859_oopAss1.cpp/22L-6194.cpp

JPlag Comparison

back

Submission 1
22L-7859_oopAss1.cpp

Submission 2
22L-6194.cpp

Match %
55.49

Submission 1 File 1	Submission 2 File 2	Tokens
22L-7859_oopAss1.cpp\22L-7859_oopAss1.cpp (3 - 37)	22L-6194.cpp\22L-6194.cpp (5 - 44)	67
22L-7859_oopAss1.cpp\22L-7859_oopAss1.cpp (41 - 62)	22L-6194.cpp\22L-6194.cpp (47 - 84)	35
22L-7859_oopAss1.cpp\22L-7859_oopAss1.cpp (81 - 105)	22L-6194.cpp\22L-6194.cpp (115 - 139)	28
22L-7859_oopAss1.cpp\22L-7859_oopAss1.cpp (69 - 75)	22L-6194.cpp\22L-6194.cpp (98 - 105)	14

Files of 22L-7859_oopAss1.cpp

..2L-7859_oopAss1.cpp\22L-7859_oopAss1.cpp

```cpp
1 #include <iostream>
2 using namespace std;
3 void tokenize(char data[], char**& DataDictionary, int& num_words) {
4     int s_index = 0;
5     bool flag;
6     for (int i = 0; i < strlen(data); i++) {
7         if (data[i] == ' ' || data[i] == '.') {
8             int length = i - s_index;
9             if (length > 0) {
10                char* word = new char[length + 1];
11                for (int j = 0; j < length; j++) {
12                    word[j] = data[s_index + j];
13                }
14                flag = false;
15                word[length] = '\0';
16                for (int j = 0; j < num_words; j++) {
17                    if (strcmp(word, DataDictionary[j]) == 0) {
18                        flag = true;
19                        break;
20                    }
21                }
22                if (!flag) {
23                    DataDictionary[num_words] = word;
24                    num_words++;
25                }
26                else {
27                    delete[] word;
28                }
29            }
30            s_index = i + 1;
31        }
32    }
33    for (int i = 0; i < num_words; i++) {
34        cout << DataDictionary[i] << endl;
35    }
36 }
37 void getSynonyms(char***& synonyms, char** DataDictionary, int num_words) {
38     synonyms = new char** [num_words];
39     for (int i = 0; i < num_words; i++) {
40         synonyms[i] = NULL;
41     }
42     for (int x = 0; x < num_words; x++) {
43         char* word = DataDictionary[x];
44         cout << "Do you want to store synonyms for " << word << "? (enter 'y' for yes
45         char choice;
46         cin >> choice;
47         if (choice == 'y') {
48             int count;
49             cout << "How many synonyms do you want to store? ";
50             cin >> count;
51             synonyms[x] = new char* [count];
52             for (int y = 1; y < count; y++) {
53                 static char input[200];
54                 cout << "Enter synonym " << y << " for " << word << ": ";
55                 cin >> input;
```

Files of 22L-6194.cpp

22L-6194.cpp\22L-6194.cpp

```cpp
1 #include <iostream>
2 #include <cstring>
3 using namespace std;
4
5 void Tokenization(char input[], char** dataDictionary,int&numWords) {
6
7     int startIndex = 0;
8     for (int i = 0; i < strlen(input); i++) {
9         if (input[i] == ' ' || input[i] == '.') {
10            int length = i - startIndex;
11            if (length > 0) {
12                char* word = new char[length + 1];
13                for (int j = 0; j < length; j++) {
14                    word[j] = input[startIndex + j];
15                }
16                word[length] = '\0';
17                // Check for duplicates
18                bool found = false;
19                for (int j = 0; j < numWords; j++) {
20                    if (strcmp(word, dataDictionary[j]) == 0) {
21                        found = true;
22                        break;
23                    }
24                }
25
26                if (!found) {
27                    dataDictionary[numWords] = word;
28                    numWords++;
29                }
30                else {
31                    delete[] word;
32                }
33            }
34            startIndex = i + 1;
35        }
36    }
37
38
39    cout << "Data Dictionary:" << endl;
40    for (int i = 0; i < numWords; i++) {
41        cout << dataDictionary[i] << endl;
42    }
43 }
44 void getsynonyms(char***synonyms,char**dataDictionary,int&numWords) {
45     for (int i = 0; i < numWords; i++) {
46         synonyms[i] = nullptr;
47     }
48
49
50     for (int i = 0; i < numWords; i++) {
51         char* word = dataDictionary[i];
52
53
54         cout << "Do you want to store synonyms for " << word << "? (y/n): ";
55         char choice;
```

11:39 AM
4/14/2023

What × | repor × | Lab M × | Down × | Fish A × | Class × | New × | Scree × | +

jplag.github.io/JPlag/comparison/22L-7859_oopAss1.cpp/22L-6194.cpp

Photos - Google Ph... | YouTube | Facebook | WhatsApp | Classes | Temp Mail - Dispos... | LESCO - Customer... | Bill Payment

JPlag Comparison

back

Submission 1
22L-7859_oopAss1.cpp

Submission 2
22L-6194.cpp

Match %
55.49

Submission 1 File 1	Submission 2 File 2	Tokens
22L-7859_oopAss1.cpp\22L-7859_oopAss1.cpp (3 - 37)	22L-6194.cpp\22L-6194.cpp (5 - 44)	67
22L-7859_oopAss1.cpp\22L-7859_oopAss1.cpp (41 - 62)	22L-6194.cpp\22L-6194.cpp (47 - 84)	35
22L-7859_oopAss1.cpp\22L-7859_oopAss1.cpp (81 - 105)	22L-6194.cpp\22L-6194.cpp (115 - 139)	28
22L-7859_oopAss1.cpp\22L-7859_oopAss1.cpp (69 - 75)	22L-6194.cpp\22L-6194.cpp (98 - 105)	14

```cpp
    }
    for (int x = 0; x < num_words; x++) {
        char* word = DataDictionary[x];
        cout << "Do you want to store synonyms for " << word << "? (enter 'y' for yes)
        char choice;
        cin >> choice;
        if (choice == 'y') {
            int count;
            cout << "How many synonyms do you want to store? ";
            cin >> count;
            synonyms[x] = new char* [count];
            for (int y = 1; y < count; y++) {
                static char input[200];
                cout << "Enter synonym " << y << " for " << word << ": ";
                cin >> input;
                synonyms[x][y] = new char[strlen(input) + 1];
                for (int z = 0; z < strlen(input); z++) {
                    synonyms[x][y][z] = input[z];
                }
                synonyms[x][y][strlen(input)] = '\0';
            }
        }
    }
}
void SearchSynonyms(char* UseCase1, char** dataDictionary, char*** synonyms, int num_w
    const int w_len = 3000;
    bool flag = false;
    char word[w_len];

    int s_index = 0, x = 0;
    cout << "Matching words found:\n";
    for (int i = 0; i <= strlen(UseCase1); i++) {
        if (UseCase1[i] == ' ' || UseCase1[i] == '.') {
            int length = i - s_index;
             if (length > 0) {
                if (length >= w_len) {
                    cout << "Error! maximum length exceeded\n";
                }
                for (int j = 0; j < length; j++) {
                    word[j] = UseCase1[s_index + j];
                }
                word[length] = '\0';
                for (int j = 0; j < num_words; j++) {
                    if (strcmp(word, dataDictionary[j]) == 0) {
                        flag = true;
                        break;
                    }
                }
                if (flag) {
                    cout << word << endl;
                    x++;
                }
            }
            s_index = i + 1;
        }
    }

    if (x == 0) {
        cout << "No Matching words found\n";
    }
    else {
        cout << "Matching words found: " << x << endl;
    }
}
void main() {
```

```cpp
    }


    for (int i = 0; i < numWords; i++) {
        char* word = dataDictionary[i];


        cout << "Do you want to store synonyms for " << word << "? (y/n): ";
        char choice;
        cin >> choice;

        if (choice == 'y' || choice == 'Y') {

            int count;
            cout << "How many synonyms do you want to store? ";
            cin >> count;

            // Allocate memory for the synonyms at this index
            synonyms[i] = new char* [count];


            for (int j = 0; j < count; j++) {

                static char temp[100];

                cout << "Enter synonym #" << j + 1 << " for " << word << ": ";
                cin >> temp;

                // Allocate memory for the synonym at this index
                synonyms[i][j] = new char[strlen(temp) + 1];

                // Copy the data character by character
                for (int k = 0; k < strlen(temp); k++) {
                    synonyms[i][j][k] = temp[k];
                }
                synonyms[i][j][strlen(temp)] = '\0';
            }
        }
        else if (choice == 'n' || choice == 'N')
        {
            synonyms[i] = new char* [1];
            synonyms[i][0] = new char[10];
            synonyms[i][0][0] = '\0';
        }
    }
}
void SearchWord(char* UseCase1, char** dataDictionary, int numWords) {
    const int WORD_LEN = 1000;
    char word[WORD_LEN];

    int startIndex = 0;

    int count = 0;

     cout << "Matched words:" << endl;

     for (int i = 0; i <= strlen(UseCase1); i++) {
         if (UseCase1[i] == ' ' || UseCase1[i] == '.') {
             int length = i - startIndex;
             if (length > 0) {
                 if (length >=WORD_LEN) {
                     cout << "Error: word length exceeds maximum length." << endl;
                     return;
                 }
                 for (int j = 0; j < length; j++) {
                     word[j] = UseCase1[startIndex + j];
                 }
                 word[length] = '\0';
```

11:39 AM
4/14/2023

What × | repor × | Lab M × | Down × | Fish A × | Class × | New × | Scree × | +

jplag.github.io/JPlag/comparison/22L-7859_oopAss1.cpp/22L-6194.cpp

Photos - Google Ph... | YouTube | Facebook | WhatsApp | Classes | Temp Mail - Dispos... | LESCO - Customer... | Bill Payment

JPlag Comparison

back

Submission 1
22L-7859_oopAss1.cpp

Submission 2
22L-6194.cpp

Match %
55.49

Submission 1 File 1	Submission 2 File 2	Tokens
22L-7859_oopAss1.cpp\22L-7859_oopAss1.cpp (3 - 37)	22L-6194.cpp\22L-6194.cpp (5 - 44)	67
22L-7859_oopAss1.cpp\22L-7859_oopAss1.cpp (41 - 62)	22L-6194.cpp\22L-6194.cpp (47 - 84)	35
22L-7859_oopAss1.cpp\22L-7859_oopAss1.cpp (81 - 105)	22L-6194.cpp\22L-6194.cpp (115 - 139)	28
22L-7859_oopAss1.cpp\22L-7859_oopAss1.cpp (69 - 75)	22L-6194.cpp\22L-6194.cpp (98 - 105)	14

```
                    synonyms[x][y][z] = input[z];
                }
                synonyms[x][y][strlen(input)] = '\0';
            }
        }

    }
}
void SearchSynonyms(char* UseCase1, char** dataDictionary, char*** synonyms, int num_w
    const int w_len = 3000;
    bool flag = false;
    char word[w_len];

    int s_index = 0, x = 0;
    cout << "Matching words found:\n";
    for (int i = 0; i <= strlen(UseCase1); i++) {
        if (UseCase1[i] == ' ' || UseCase1[i] == '.') {
            int length = i - s_index;
             if (length > 0) {
                if (length >= w_len) {

                    cout << "Error! maximum length exceeded\n";
                }
                for (int j = 0; j < length; j++) {
                    word[j] = UseCase1[s_index + j];
                }

                word[length] = '\0';
                for (int j = 0; j < num_words; j++) {
                    if (strcmp(word, dataDictionary[j]) == 0) {
                        flag = true;
                        break;
                    }
                }
                if (flag) {
                    cout << word << endl;
                    x++;
                }

            }
            s_index = i + 1;
        }
    }

    if (x == 0) {
        cout << "No Matching words found\n";
    }
    else {
        cout << "Matching words found: " << x << endl;
    }
}
void main() {

    char data[3000];
    int num_words = 0;
    char** DataDictionary = new char* [1000];
    char*** synonyms = new char** [num_words];
    cout << "Enter text (3-4 sentences) End each sentence with a fullstop: ";
    cin.getline(data, 3000);
    tokenize(data, DataDictionary, num_words);
    getSynonyms(synonyms, DataDictionary, num_words);
    char* UseCase1 = new char[3000];
    cout << "Enter text (end with fullstop): ";
    cin.ignore();
    cin.getline(UseCase1, 3000);
    SearchSynonyms(UseCase1, DataDictionary,synonyms, num_words);
}
```

```
                bool found = false;
                for (int j = 0; j < numWords; j++) {
                    if (strcmp(word, dataDictionary[j]) == 0) {
                        found = true;
                        break;
                    }
                }
                if (found) {
                    cout << word << endl;
                    count++;
                }

            }
            startIndex = i + 1;
        }
    }

    if (count == 0) {
        cout << "No words found in data dictionary." << endl;
    }
    else {
        cout << "Total matched words: " << count << endl;
    }
}
void replaceSynonyms(char*** synonyms, char** dataDictionary, int wordCount)
{
    bool isPresent = true;
    char* useCase1 = new char[2500];
    char* temp = new char[2500];
    int i = 0,j=0,forTemp=0;
    cout << "enter sentence(s) to replace synonyms: ";
    cin.getline(useCase1, 2500);
    while (useCase1[i] != '\0')
    {
            if (i == 0 || useCase1[i] == ' ' || useCase1[i] == '.')
            {
                    if (i != 0)
                            i++;
                    int temp1 = i;
                    j = 0;
                    while (j < wordCount)
                    {
                            int characters = 0;
                            temp1 = i;
                            while (useCase1[temp1] != ' ' && useCase1[temp1] != '.' &
                            {

                                    if (useCase1[temp1++] == dataDictionary[j][charac
                                    {
                                            isPresent = true;
                                    }
                                    else
                                    {
                                            isPresent = false;
                                            break;
                                    }
                            }
                            if (isPresent)
                            {
                                    if (synonyms[j][0][0] == '\0')
                                            isPresent = false;
                                    break;
                            }
                            j++;
                    }

                    if (isPresent)
                    {
                            int h = 0;
```

11:39 AM
4/14/2023